

General disclosure

- GPT 6 sol high for implementation and astra med for auditing
- Both were consulted on 9/22, I had the sol high chat handle implementation and testing and told it to spawn an astra subagent for auditing.

Code attribution

- Please see the prompt chain at the bottom of the doc
- All Ai generated except for prologue comments made (showing my understanding of the code)
- Made no modifications. As for “a disclosure that shows what you asked, what came back wrong, and how you diagnosed” I looked through some of the tests, did a visual review of the code but had no issues with bad output from the model. I expressed some uncertainty (as you can see in the last prompt I sent) but there was nothing problematic found.

Verification statement

- I understand every line of the submitted code.
- The code has been tested and works as intended.
- No known limitations.

Prompts sent numbers below:

- 1) I fed the llm this prompt: please read these instructions for an llm assisted coding/documenting task. Once you are familiar help me build an understanding of the project, do not implement anything this is an exercise to help me understand better:
“**Problem description** Write a C++ program that reads a line of text and extracts a single valid IPv4 address — optionally followed by a port number — embedded anywhere in that text. Only digits, periods (.), and colons (:) are ever part of a valid token; every other character is garbage and is skipped. A candidate token must match the address grammar **in full** — no partial matches, no truncating to find a valid piece inside a longer run. An address is four octets separated by periods (octet.octet.octet.octet), each octet 1–3 digits, value 0–255, no leading zero unless the value is exactly 0. An optional :port may follow the fourth octet: 1–5 digits, value 0–65535, same leading-zero rule. If a colon is present, the port must be fully valid or the entire match — address included — is rejected. **Function prototype** The function returns whether a valid address was found, and delivers the address and port through two additional parameters. C++//
Returns true if a valid address was found, false otherwise.
// On success: outAddress holds the 32-bit value,
// and outPort holds the port number, or -1 if no port was present.
// On failure: outAddress is set to 0 and outPort is set to -1.
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort); **Other requirements**
 - Exactly one valid address may be extracted per input line; everything else in the line is either garbage (skipped) or part of a candidate token that fails validation.

- Reject anything that does not exactly match the grammar above — wrong octet count, empty octet, out-of-range octet or port, a disallowed leading zero, a second colon, a colon not immediately after the fourth octet, or a stray period/colon directly adjacent to an otherwise-valid address.
- On success, print: `Extracted IPv4 address: A.B.C.D (decimal value: N, port: P)` where N is the 32-bit decimal value and P is the port number or the literal text none.

2) Great I understand the task lets proceed.here is more info about the project: **Built-in functions/libraries you may not use**

- Any string-to-number conversion function: `atoi`, `atol`, `atoll`, `strtol`, `strtoul`, `strtod`, `stoi`, `stol`, `stoul`, `sscanf`, `scanf` with numeric conversions.
- Any address-parsing library function: `inet_aton`, `inet_pton`, `inet_addr`, or equivalents.
- Any regular-expression facility (`std::regex`, `POSIX regex.h`, or similar) — the parsing and validation logic must be your own character-by-character code, not a pattern matched by a library. Standard character-classification functions (`isdigit`, etc.) are fine to use. **Program requirements**
- **Input loop:** `main` continuously prompts the user for input until the user enters END (case-sensitive), then prints Program terminated. and exits.
- **Extraction function:** implement `extractIPv4` exactly as prototyped above. All digit accumulation must be done by hand. **Display:** `main` receives the result from `extractIPv4` and formats the output exactly as specified above.”, please turn the rules into a small set of test cases before writing the parser. ive changed to gpt 6 sol high, first reread the instructions to make sure you have a clear picture of the assignment. from there please fully implement in cpp, run tests after implementation and document if any issues in the initial implementation phase. after this is done use a astra med subagent to audit and make sure all instructions requirements are satisfied

(Model came back saying everything passed so I continued to manual review)

- 3) Please check my IPv4 parser, im concerned about if it accidentally accepts part of a malformed address when there's an extra period or colon, or invalid port. Look over the cpp files and the tests. then run make test and try a few edge cases. I want the smallest failing input with expected/actual results if it doesnt all pass

(Model came back saying: Astra Medium completed the final audit and found **no genuine failing edge case**. The existing tests passed, and its 4,350 additional probes passed.)